SCALECART · DISTRIBUTED ORDER PROCESSING PLATFORM

Interview Questions & Answers

Parts 1–6 · 70 questions with full answers, based on what was actually built

Repository: github.com/MayurGaikwad57/Scale-Cart-Learning

Covers milestones M1–M6, all built and merged · November 2026

Contents

Part 1 — M1 Foundation (architecture, database design) Q1–Q8

Part 2 — M2 Backend Core (auth, products, inventory, cart, orders, transactions) Q9–Q18

Part 3 — M3 Frontend Core (Angular 21) Q19–Q27

Part 4 — M4 Performance (own Redis "ScaleKV", caching, rate limiting, idempotency) Q28–Q39

Part 5 — M5 Kafka (own Kafka "ScaleLog", events, consumer groups) Q40–Q51

Part 6 — M6 Microservices (API gateway, payment, inventory, notification services) REWRITTEN · BUILT Q52–Q70

How to use this document: read the question, answer it out loud first, then compare with the written answer.

Parts 1–3 were written from what was built (the plan's "challenge" step had been skipped for them); Parts 4–5 include the question lists from `docs/06-performance.md` and `docs/07-events.md` word for word, plus the example questions from `project-plan.md` §25.

New in this edition: Part 6 is no longer a plan. It describes the microservices that were built in M6 (PR #4): the 4 questions from `project-plan.md` §25, the 10 from `docs/08-microservices.md`, and 5 more on what the code review and the tests taught. Green boxes in earlier parts point out where M6 changed an answer.

PART 1 · MILESTONE M1

Foundation

Architecture and database design

1 What is ScaleCart, in one minute?

An e-commerce order-processing platform I built to learn distributed systems step by step. It started as a modular monolith: one Node/Express + TypeScript backend with clear modules (users, products, inventory, cart, orders, payments) on PostgreSQL, plus an Angular 21 frontend. Each phase adds one production concern: caching and rate limiting (M4), event streaming (M5), then microservices, reliability patterns, scaling and observability. Under a no-install constraint, I built my own Redis-like store (ScaleKV) and my own Kafka-like broker (ScaleLog).

> Since M6 it runs as an API gateway in front of four services (order, payment, inventory, notification) — see Part 6.

2 Why start with a modular monolith instead of microservices?

Microservices add network failures, distributed transactions, deployment and debugging cost from day one. A monolith gives one database transaction for checkout, simple deployment and fast refactoring. Modular means each module has its own routes, service and repository, and modules talk through service functions rather than each other's tables. So when a module needs to become a service later (payments in M6), the boundary already exists. You split when you have a reason, such as independent scaling, team ownership or failure isolation, not by default.

3 Why store money as integer cents instead of a decimal/float?

Floating point can't represent values like 0.1 exactly: 0.1 + 0.2 ≠ 0.3, so totals drift. Integer cents (`price_cents INT`) make every sum exact, and formatting to ₹ happens only at display time. (DECIMAL in the database also works, but integers are simpler to move between DB, JSON and JavaScript without precision loss.)

4 Why UUIDs as primary keys instead of auto-increment integers?

UUIDs can be generated anywhere (any service, any shard) without asking a central counter, so they survive moving data across services or shards later. They also don't leak business volume: order #1042 tells a competitor how many orders you have. The cost is bigger indexes and random insert order (B-tree page splits). Time-ordered UUIDs (v7) reduce that.

5 Which indexes did you create and why?

Each index exists for a specific query:

- `products (category, price_cents)` for the catalog filter by category plus price sort/range.
- `orders (user_id, created_at DESC)` for "my orders, newest first", so the database reads the index in order instead of sorting.
- `UNIQUE cart_items (cart_id, product_id)`: one line per product per cart, and it doubles as the index for "items of a cart".
- `UNIQUE users(email)`, `UNIQUE products(sku)` for lookups and duplicate prevention.
- `order_items(order_id)`, `payments(order_id)` for joining an order to its lines and payments.

6 What constraints protect data integrity in the database itself?

Rules the application could forget are enforced by PostgreSQL:

- `CHECK (available ⩾ 0)` on inventory: stock can never go negative, even if the code has a bug.
- `CHECK (quantity > 0)` on cart items.
- Foreign keys between all tables.
- Unique constraints on email, SKU, cart line, and `(user_id, idempotency_key)` on orders.

The database is the last line of defence.

7 Why is inventory a separate table from products?

Products are read-heavy catalog data; inventory is write-heavy, contended data (every checkout locks and updates it). Separating them means a checkout's row lock on stock doesn't block reads or edits of product descriptions. It also prepares an independent Inventory service later. The table also has `reserved` (stock held during payment) and `version` (for optimistic locking).

> That preparation paid off: in M6 the inventory table moved to its own Inventory Service unchanged.

8 What did you NOT finish in M1, honestly?

The Docker setup (`docker-compose.yml`) was written but never tested, because Docker can't be installed on the development machine. I run PostgreSQL as a local service instead. In an interview it's better to say this than to pretend it was verified.

PART 2 · MILESTONE M2

Backend Core

Auth, products, inventory, cart, orders, transactions

9 How does authentication work?

On register, the password is hashed with bcrypt (slow, salted hashing, so a leaked database can't be reversed quickly) and stored as `password_hash`. On login, bcrypt compares the hash and the server signs a JWT containing the user id (`sub`) and role, with an expiry. The client sends `Authorization: Bearer <token>` on every request, and the `authenticate` middleware verifies the signature and attaches `req.user`. `requireRole("ADMIN")` guards admin routes. JWTs are stateless (no session lookup), but you can't revoke one before it expires without extra machinery (short expiry + refresh tokens, or a deny-list).

10 Walk me through checkout. Why is it one transaction?

Inside one database transaction:

1. Lock the user's cart row (`SELECT … FOR UPDATE`).
2. Read the cart and reject an empty cart or inactive products.
3. Lock the stock rows (`FOR UPDATE … ORDER BY product_id`).
4. Check availability.
5. Move stock from `available` to `reserved`.
6. Create the order with a price snapshot per line.
7. Charge the (simulated) payment.
8. On success, confirm the order, turn the reservation into a sale and empty the cart. On decline, compensate: give the reserved stock back, mark the order CANCELLED, keep the cart.

Any error rolls back everything. You can never end up with an order but no stock change, or stock taken but no order.

> This was the M2–M5 design. In M6 the stock and the payment moved to other services, so one transaction is no longer possible — checkout became a sequence of local steps with compensations (Q57).

11 Two customers buy the last item at the same moment. What happens?

Both transactions reach `SELECT … FROM inventory … FOR UPDATE`. PostgreSQL gives the row lock to one of them; the other waits. The first sees `available = 1`, reserves it and commits. Only then does the second get the lock, and it reads the current value 0 and fails with `insufficient_stock`. Without the lock, both could read 1 (a race condition) and oversell. The seeded "Limited Edition Smartwatch" with stock 1 exists to demonstrate this.

> Still true in M6: the same lock now runs inside the Inventory Service's `reserve()`, and the concurrency tests pass through the gateway.

12 Why `ORDER BY product_id` when locking stock rows?

To prevent deadlocks. If cart A locks product 1 then 2, while cart B locks 2 then 1, each can hold one lock and wait forever for the other. If every transaction always locks rows in the same global order (sorted by id), a cycle of waiting is impossible.

13 Pessimistic vs optimistic locking — which did you use and why?

Pessimistic (`SELECT … FOR UPDATE`): lock first, then read and modify; others wait. Optimistic: read without locking, then update with `WHERE version = :readVersion`; if zero rows changed, someone else won, so retry. I used pessimistic for checkout because stock on a popular item is a high-contention row, where optimistic locking would cause many failed retries. Optimistic suits low contention (e.g. two admins rarely editing the same product). The `inventory.version` column is there for that comparison.

14 Why snapshot the price on each order item?

Prices change. If an order only referenced the product, raising the price tomorrow would rewrite last month's order totals and invoices. `order_items.unit_price_cents` freezes what the customer actually paid. History must be immutable.

15 Why soft-delete products instead of deleting rows?

Order items reference products through foreign keys, and past orders must stay readable. A hard delete would either fail (FK violation) or cascade and destroy order history. `active = false` hides the product from customers (it returns 404 for them) while admins still see it, and old orders still show its name.

16 Why does another user's order return 404 instead of 403?

403 Forbidden confirms the order exists, which leaks information (an attacker can enumerate valid order IDs). The query filters by `id AND user_id`, so someone else's order is simply "not found". This is a common defence against insecure direct object references (IDOR).

17 How do you validate input and handle errors consistently?

Every request body is validated with zod schemas in a `validateBody` middleware. Invalid input becomes `400 validation_failed` with per-field details. Expected failures throw an `HttpError(status, code, message)`. A single error-handler middleware turns everything into the same JSON shape (`{ error, message?, details? }`), and unexpected errors are logged but returned as `500 internal_error` without leaking internals. Config is also validated at boot (`env.ts`), so the app fails fast with a clear message.

18 How did you test the concurrency logic?

Integration tests run against the real PostgreSQL through the HTTP API (supertest). They include concurrent checkouts racing for the last unit, and assert that exactly one wins and stock never goes negative. Each test run tags its data with a unique prefix and cleans up only what it created. I also mutation-checked the tests: deliberately removing the lock or the check makes a test fail. That proves the tests actually detect the bug.

PART 3 · MILESTONE M3

Frontend Core

Angular 21: standalone, zoneless, signals

19 What is “modern Angular” in this project?

- Standalone components: no NgModules.
- Zoneless change detection: no zone.js; Angular updates the view when signals change.
- Signals for state.
- New control flow (`@if` , `@for` , `@let`).
- Lazy-loaded routes with `loadComponent` , so every page’s code downloads only on first visit.
- OnPush on every component.
- Route params bound as component inputs (`withComponentInputBinding`).

20 Signals vs RxJS — how did you decide which to use where?

Signals are for state: synchronous values the UI reads (logged-in user, cart, notifications), with `computed()` for derived values (cart count and total can never disagree with the items). RxJS is for async streams over time: HTTP calls, debouncing the search box (`debounceTime`), cancelling stale requests (`switchMap`), polling with `timer` . The two meet through `toSignal()` . The rule from the plan: don’t use signals just because they’re new; pick per piece of state.

21 Where does each piece of state live?

- Logged-in user and cart: signals in root services, because the navbar, product pages, cart and checkout all need them at once.
- Reloading the cart on login/logout: an `effect()` .
- Form inputs: Reactive Forms (validation, touched/dirty).
- Loading/error flags, menus: local component signals.
- Session across reloads: localStorage + `/auth/me` at startup.

22 How does the HTTP interceptor work?

On every request to our API (and only our API, never third parties) it adds `Authorization: Bearer <token>` . If the API answers 401 on a protected call, the session is dead, so it logs out and redirects to `/login?returnUrl=…` . The login and register calls themselves are excluded, because a 401 there means “wrong password”, not “expired session”.

23 Are route guards security?

No. `authGuard` , `adminGuard` and `guestGuard` are UX only: they stop people from seeing pages they can’t use. Anyone can call the API directly, so the API enforces authorization (`authenticate` , `requireRole`). Never trust the client.

24 What is an open redirect, and how did you prevent it?

After login we send the user back to `returnUrl`. If an attacker crafts `…/login?returnUrl=https://evil.com`, a naive app would forward a freshly logged-in user to a phishing site. `safeReturnUrl()` only allows in-app paths starting with a single `/`, and rejects absolute URLs and protocol-relative `//evil.com`. It's unit-tested.

25 Storing the JWT in localStorage — what's the trade-off?

It's simple and survives reloads, but any JavaScript running on the page can read it, so an XSS bug could steal the token. The hardened alternative is an httpOnly, Secure, SameSite cookie (JavaScript can't read it), which then needs CSRF protection. For a learning project I chose simplicity and documented the trade-off.

26 How does the product search avoid flooding the API while the user types?

Search, filter and page live in signals. A `computed()` builds the query, and RxJS adds `debounceTime` (wait until typing pauses) and `switchMap` (cancel the previous in-flight request when a new query starts, so an old slow response can't overwrite a newer one). The result is exposed via `toSignal`.

27 How did you verify the UI really works?

Unit tests for services and helpers (vitest), plus browser journeys driven in real Edge (headless, via puppeteer-core): login, add to cart, search, checkout declined and successful, order history, logout guard, admin product creation. I took screenshots of every page at desktop width and at 390px phone width. Testing on a phone width caught a real bug later (a notification dropdown cut off on the left that a "no horizontal scroll" check couldn't see).

PART 4 · MILESTONE M4

Performance

Own Redis “ScaleKV”, caching, rate limiting, idempotency

FROM DOCS/06-PERFORMANCE.MD

28 Why must a cache or rate limiter be a separate process when you run several API instances?

In production several copies of the API run behind a load balancer. A `Map` inside one process is invisible to the others:

- Cache: each instance has its own copy, and an invalidation reaches only the instance that handled the write, so the others keep serving stale data.
- Rate limiter: each instance counts separately, so a client spread over N instances gets N× the limit.

Shared state has to live outside the app processes, which is exactly why Redis exists. That’s why ScaleKV is its own process that every API instance talks to over TCP.

29 Redis is single-threaded. Why is that fast, and why does it make `INCR` safe?

Each command is tiny in-memory work (microseconds), so the bottleneck is the network, not the CPU. A single thread means no locks, no lock contention and no context switches. And because commands run one at a time to completion, `INCR` (read, add 1, write) can never interleave with another client’s `INCR`. That makes it atomic for free. ScaleKV works the same way: Node runs JavaScript on one thread and `execute()` never awaits mid-command.

30 Walk through cache-aside. Why delete after the DB commit and not before? What race remains?

Read: look in the cache. On a HIT, return it. On a MISS, read PostgreSQL, store the result with a TTL, return it. **Write:** update PostgreSQL first, then invalidate.

If you invalidate before the commit, a reader can slip in between, read the old row (not yet committed) and cache it again, and the stale value lives for the full TTL. A race remains even with delete-after-commit: a reader misses and reads the old row; the writer commits and deletes; then the slow reader stores the old row. My fix (found in code review) is versioned keys. Every product has a version counter (`product:ver:<id>`), the cache key includes it (`product:<id>:v<n>`), and a write bumps the counter. The slow reader’s late write lands on an old version that nobody reads anymore. TTL bounds any remaining staleness.

31 Explain stampede vs avalanche vs penetration and one fix for each.

- **Stampede** (thundering herd): a hot key expires and hundreds of requests miss at once and all hit the database. Fix: single-flight: concurrent misses for the same key share one database query (across instances: a short `SET lock NX PX` mutex, or refresh before expiry).
- **Avalanche:** many keys created together expire together, and the database gets the whole load at once. Fix: TTL jitter (±10% random), so expiries spread out.
- **Penetration:** requests for IDs that don’t exist always miss and always hit the database (often an attack). Fix: negative caching: cache “not found” briefly (15s), plus validating the UUID format before any lookup (at scale: a Bloom filter).

32 Fixed window vs sliding window counter vs token bucket: which would you pick for login, and for a public API?

- **Fixed window:** one counter per minute. Cheap, but allows a 2× burst at the boundary (100 requests at 12:00:59 and 100 more at 12:01:00).
- **Sliding window log:** a timestamp per request. Exact, but memory grows with the limit.
- **Sliding window counter** (what I built): `estimate = previous × (1 - elapsed fraction) + current`. Two small counters, no boundary burst, only needs `INCR` / `GET`.
- **Token bucket:** a refill rate plus a capacity. Allows controlled bursts and needs an atomic read-modify-write (a Lua script in Redis).

For login, use a strict per-IP/per-account sliding window with no bursts (brute force is the threat), and possibly fail closed at a bank. For a public API, a token bucket lets well-behaved clients burst a little while capping the average. Mine fails open: if ScaleKV is down, requests are allowed, because a cache outage shouldn't become a full site outage.

> M6 twist: behind a gateway, the per-IP limit depends on the services seeing the real client IP — and a code-review bug showed how a faked `X-Forwarded-For` could defeat it (Q69).

33 A client retries `POST /orders` after a timeout. How do you guarantee one order? What if Redis lost the key?

The client sends an `Idempotency-Key` (one per purchase attempt). There are two layers:

1. **ScaleKV (fast layer):** `SET idem:order:<user>:<key> {running, fingerprint} NX PX 60s`.
   - Another request is still running with that key: `409 request_in_progress`.
   - The first request finished: replay the stored response with `Idempotent-Replayed: true`.
   - Same key with a different body (SHA-256 fingerprint): `422 idempotency_key_reused`.
   - The handler failed: delete the record so the key can be retried.
2. **PostgreSQL (the guarantee):** `UNIQUE (user_id, idempotency_key)` on orders plus the stored fingerprint.

If ScaleKV lost the key (restart, eviction, outage), the database still protects you. Inside the checkout transaction, after locking the user's cart row, we look for an order with that key ("lock, then check", so no race), and the unique constraint is the final backstop. The frontend keeps the same key on unknown outcomes (network error, 5xx, 409 in progress) and retries automatically, which is safe only because of the key.

34 Your client library times out a command but the reply arrives later. What can go wrong?

On a pipelined connection, replies are matched to commands purely by order (FIFO). If the timed-out command's slot is removed from the queue, its late reply gets handed to the next command. That command returns someone else's data, which is a silent, nasty bug. The fix: keep the slot but mark it settled, and drop its late reply when it arrives. A test reproduces head-of-line blocking to prove it. Separately, a circuit breaker treats 3 timeouts in a row as "server frozen": it drops the connection, fails fast and reconnects with backoff.

FROM PROJECT-PLAN.MD §25, "AFTER REDIS"

35 Why Redis?

For shared, fast, in-memory state across all API instances: a cache in front of PostgreSQL (fewer and faster reads), rate-limit counters, and idempotency records. It gives atomic operations (`INCR`, `SET NX`), TTLs, and eviction when memory is full. In ScaleCart I couldn't install Redis, so I built ScaleKV, which speaks Redis's real RESP protocol. Switching to real Redis would only mean changing `KV_URL`.

36 Why Cache Aside?

The application controls caching explicitly. Only data that's actually requested gets cached (lazy loading), the cache can be down without breaking anything (we just read the database: `X-Cache: BYPASS`), and invalidation stays in our code next to the writes. Alternatives: write-through (write to cache and DB together; always warm, but caches data nobody reads), write-behind (write to cache, flush to DB later; fast, but risks data loss), read-through (the cache loads from the DB itself; needs cache-side loaders).

37 What happens when Redis goes down?

In ScaleCart the app degrades instead of failing:

- Commands fail fast (no waiting for timeouts).
- Product reads go straight to PostgreSQL (`BYPASS`, slower but correct).
- Rate limits fail open.
- Idempotency falls back to the database layer.
- The client reconnects in the background with exponential backoff (100ms → 5s), and the admin System page shows "Down".

I demonstrated this live by killing ScaleKV mid-use: the shop kept working and reconnected by itself when ScaleKV came back.

38 What happens during cache stampede?

A popular key expires, every concurrent request misses at the same moment, and they all query the database together. Database load spikes, latency rises, and timeouts can cascade. Mitigations: single-flight (I implemented it per process), a distributed lock so only one instance recomputes, early/probabilistic refresh before the TTL ends, and TTL jitter so keys don't all expire at once.

39 How would you invalidate the cache?

- **Single items:** after the database commit, invalidate the product's key. I bump a per-product version counter (versioned keys) so a late stale write is unreachable.
- **Many keys at once** (all catalog list pages, one per filter/sort/page combination): never `KEYS products:*` (O(N), and it blocks Redis's single thread). Instead `INCR products:ver`; every list key includes the version, so all old pages become unreachable at once and expire via their TTL.
- Checkout refreshes only the sold products' pages, so the list cache keeps its hit rate under steady orders (trade-off: list-page stock may lag up to 30s, while the product page and checkout are always exact).
- At scale: CDC-driven invalidation (M8).

> In M6 the Inventory Service invalidates a product's page whenever `available` changes: on reserve and on release (not on commit, which only moves units from reserved to sold).

PART 5 · MILESTONE M5

Kafka

Own Kafka “ScaleLog”, events, consumer groups

FROM DOCS/07-EVENTS.MD

40 Kafka vs a queue (RabbitMQ/SQS): what changes when consuming doesn’t delete?

In a queue, a consumed and acknowledged message is removed, and one message goes to one consumer. In Kafka the topic is an append-only log: reading doesn’t delete, and each consumer group keeps its own position (offset). So:

- Many independent groups can each read every event (notification, inventory and analytics all see every `OrderCreated`).
- You can replay history by moving an offset.
- A new consumer can be added later and process old events.
- Progress is a single number per partition instead of per-message acks.
- Retention is time- or size-based, not consumption-based.

> M6 used exactly this: the new `payment` group was added to the existing `orders` topic without changing any producer.

41 How does Kafka keep events of one order in order, and what does that cost?

Order is only guaranteed within a partition. The producer uses a key (the order ID), and `hash(key) % partitions` sends all events of that order to the same partition. One partition is processed by one group member, sequentially, so per-key order holds. The costs:

- There’s no global order across partitions.
- A group’s parallelism is capped by the partition count (4 members on 3 partitions means one is idle).
- A hot key overloads its partition.
- Changing the partition count remaps keys, which breaks ordering during the change.

42 A consumer crashes after writing to the DB but before committing. What happens, and how do you make it safe?

The offset wasn’t committed, so after the rebalance the event is delivered again (to the restarted consumer or another member). That’s at-least-once delivery: commit after handling. (Committing before handling would be at-most-once, where a crash loses the event.) To make it safe, consumers must be idempotent. In ScaleCart, each handler runs inside `once(group, eventId, …)`, which inserts `(consumer_group, event_id)` into `processed_events` in the same database transaction as the side effect. Either both commit or both roll back, and a redelivery hits the primary key and is skipped. That turns at-least-once delivery into exactly-once effect. A tested subtlety from code review: only a conflict on that marker means “duplicate”; any other error must be retried, or the event is silently lost.

43 Explain a rebalance. What is a zombie consumer and how is it stopped?

Group members send heartbeats (every 1s in ScaleLog). When a member joins, leaves (graceful LEAVE) or misses heartbeats for the session timeout (10s), the broker reassigns the partitions (range assignment) and bumps the group's generation number. Measured live: after a hard kill, the API took over in 10.3s. A zombie is a member that was declared dead (e.g. it froze for a GC pause or a network blip) but is still running and tries to commit for a partition that now belongs to someone else. Fencing stops it: every commit carries the member's generation, and the broker rejects a stale generation (`ILLEGAL_GENERATION`) or a partition the member doesn't own (`NOT_ASSIGNED`). The zombie then re-reads its assignment, and its uncommitted event is redone by the new owner (a duplicate, which idempotency absorbs).

44 One event always fails. What happens without a DLQ, and what does a DLQ cost you?

Without a DLQ, the consumer retries that poison message forever. Since a partition is processed in order, everything behind it is stuck: head-of-line blocking. Lag grows and customers stop getting notifications. With a DLQ, after N retries with exponential backoff (ScaleLog: 3 attempts, 200ms/400ms) the record is copied to `orders.<group>.dlq` with the error, the attempt count and its original location; the offset is committed and the partition keeps flowing. If even the DLQ write fails, we don't commit, so nothing is lost. The cost is ordering: while an event waits in the DLQ, later events for the same order can overtake it, so a replay can apply things out of order. There's also operational work: someone must inspect and replay (I built an admin Replay button, plus a "chaos switch" to demonstrate it).

45 Your order is saved but the event wasn't published. Why can this happen and how do you fix it?

It's the dual-write problem. Checkout writes the order to PostgreSQL and then sends the event to the broker: two systems with no shared transaction. If the broker is down, or the process dies between the two writes, the order exists but its event never does, so there's no notification and no analytics. ScaleCart doesn't fail the checkout for this; it counts missed events and shows a warning on the admin Events page. The fixes:

- **Transactional outbox (M7):** write the event into an outbox table in the same transaction as the order; a relay process publishes from the outbox and marks rows as sent (at-least-once again, so consumers stay idempotent).
- **CDC (M8):** stream the database's own change log (WAL) into the broker.

> In M6 a lost event matters more (payment now depends on it), so I added a reconciler and an inventory sweeper as safety nets until the outbox arrives (Q62).

46 What does "consumer lag" tell you and when do you alert on it?

Lag = log end offset − committed offset, per partition: events written but not yet processed by that group. Some lag that stays steady is normal. Lag that keeps growing means consumers can't keep up (too slow, crashing, stuck on a poison message, or too few partitions/members). Alert on sustained growth or on lag measured in time (e.g. "notifications more than 2 minutes behind"), not on a single spike. The admin Events page shows lag per group and partition.

FROM PROJECT-PLAN.MD §25, "AFTER KAFKA"

47 What happens when a consumer crashes?

Its heartbeats stop. After the session timeout (10s) the broker removes it, rebalances, and gives its partitions to the remaining members (new generation). They resume from the last committed offset, so any event the crashed consumer handled but didn't commit is processed again. Nothing is lost (at-least-once), and duplicates are absorbed by idempotent handlers. A graceful stop (Ctrl+C) sends LEAVE, so the rebalance is immediate instead of waiting 10s.

48 What happens before offset commit?

The consumer fetches records from its position, runs the handler for one record (with retries; on final failure it writes the record to the DLQ), and only then commits `offset + 1`. Until that commit succeeds, the broker still considers the record unprocessed. If the consumer dies at any point before the commit, the record is redelivered. If the commit is rejected (fenced after a rebalance), the consumer drops its position and refreshes its assignment.

49 How do you handle duplicates?

Assume they will happen: redelivery after a crash or rebalance, producer retries, and DLQ replays all cause them. Every event carries a unique `eventId`. Each consumer group records processed IDs in `processed_events`, in the same transaction as its work, so the second delivery is a no-op. Some effects are naturally idempotent (an upsert setting a value); others are not (`revenue += total`), and those must be guarded. I tested it by re-appending an already-processed event: still one notification, and revenue counted once.

50 What causes rebalancing?

- A member joins (a new worker starts; `npm run worker` triggers it).
- A member leaves gracefully (shutdown sends LEAVE).
- A member misses heartbeats past the session timeout (crash, freeze, long GC pause, network partition).
- In real Kafka also: a change in the topic's partitions, or consumers exceeding `max.poll.interval` (taking too long between polls).

Rebalances pause progress briefly and can cause duplicates, so you avoid unnecessary ones (stable membership, sensible timeouts, cooperative rebalancing in real Kafka).

51 How do partitions affect scaling?

A partition is the unit of parallelism: within one group, each partition is consumed by exactly one member. So the maximum useful consumers in a group = the number of partitions (3 partitions means at most 3 busy members; a 4th sits idle, as a test shows). More partitions mean more parallelism and throughput, but also more files, more open handles, longer rebalances, and ordering only per key. Increasing partitions later changes key-to-partition mapping, so choose the count with growth in mind.

PART 6 · MILESTONE M6

Microservices

API gateway + payment, inventory and notification services — built and merged (PR #4)

→ The architecture in one picture (say this first)

```
Angular :4300 ─► API Gateway :3000 ─┬─► Order :3001         users, catalog, cart, orders
                (own reverse proxy) ├─► Payment :3002       payments (from events)
                                    ├─► Inventory :3003     stock + reservations
                                    └─► Notification :3004  the customer's bell

Events through ScaleLog:
  orders topic    OrderCreated, OrderConfirmed, OrderCancelled
  payments topic  PaymentSucceeded, PaymentFailed
```

Each service is its own Node process with its own port and its own consumer groups. The browser still talks to one address. Under the no-install constraint there is no Docker/Kubernetes/nginx: the gateway is ~80 lines of my own reverse proxy, and `npm run dev` starts all seven processes (ScaleKV, ScaleLog, four services, gateway).

FROM PROJECT-PLAN.MD §25, "AFTER MICROSERVICES"

52 Why did you extract Payment?

Payment has the most independent failure characteristics: it talks to an external provider that is slow (I simulate 1.2 s), sometimes down, rate-limited, and has its own retry/timeout/idempotency rules and compliance scope (card data, PCI). In the monolith, a slow provider kept the whole checkout transaction — and its row locks on the cart and the stock — open while we waited. As its own service, consuming `OrderCreated` from the log, a provider outage no longer blocks checkout: orders simply wait in PENDING and are charged when it recovers. The boundary was already clean (`paymentsService.charge`), so extraction was cheap. Inventory came next because stock is the most contended data, and Notification because it is a pure reaction to events.

53 Why not extract everything?

Every extraction adds network calls that can fail or time out, duplicated data, compensations instead of one SQL transaction, more processes to run and debug, and versioned contracts between teams. Users, catalog and cart had no driver to move (no different scaling, ownership or failure profile), so they stayed in the Order Service — the remaining monolith. That is the strangler-fig approach (Q64): extract when there is a reason. Splitting everything at once creates a "distributed monolith": services that must change and deploy together, with all of the costs and none of the benefits.

54 How do services communicate?

Two styles, chosen per interaction:

- **Synchronous HTTP** when the caller needs the answer now: the Order Service calls the Inventory Service's `POST /internal/reservations` during checkout, because the customer must hear "out of stock" immediately. Always with a timeout (3 s) and a service token (`X-Internal-Token`).
- **Asynchronous events** through ScaleLog when a service only announces a fact: `OrderCreated` → Payment; `PaymentSucceeded/Failed` → Order; `OrderConfirmed/Cancelled` → Inventory, Notification, Analytics. Loose coupling: the producer doesn't know who listens, and consumers can be down.
- **Browser → gateway → service** over HTTP. `/internal/*` routes are never forwarded by the gateway.

Rule of thumb: synchronous for questions, events for facts. Every synchronous call is a coupling in availability, so I kept exactly one in the checkout path.

55 What happens when one service is unavailable?

Built and tested for each one:

Service down	What the customer sees	Why
Payment	Checkout still works; the order stays "processing payment", then completes when it's back.	Async: `OrderCreated` waits in the log.
Inventory	"Try again in a moment", nothing charged, cart kept. Product pages still load.	Sync call refused → we know nothing was reserved → undo.
Notification	Only the bell fails (502 for `/notifications`); checkout unaffected; messages arrive later.	Async consumer; gateway isolates its routes.
Order	Most of the site is down (it is still the core).	The honest weak spot; it would be the next to split / replicate.

The gateway turns a dead service into a quick `502` and a slow one into `504` instead of hanging, and `/health/ready` reports "degraded" with which service is down. The goal: one failing service degrades one feature, not the shop. Next (M7): retries with backoff and a circuit breaker on the reserve call.

FROM DOCS/08-MICROSERVICES.MD

56 Why is payment asynchronous but stock reservation synchronous?

They have opposite profiles. **Reservation** is fast, internal and contended, and its answer changes what the customer does next ("only 1 left"); doing it asynchronously would mean accepting orders and cancelling them later with an apology e-mail — acceptable at Amazon scale, poor UX here. **Payment** is slow, external and flaky, and the customer can wait a second for it: checkout answers `201 PENDING` in ~150 ms and the order page polls until CONFIRMED or CANCELLED ("processing your payment…"). Stock is reserved first, so it can't be sold twice while the payment is in flight.

57 Walk me through the checkout now that it spans three services.

1. **Order service, one local transaction:** lock the cart, idempotency check, create the order `PENDING` with price snapshots, and move the cart into the order (a double-click's second request then sees

an empty cart).

2. **Inventory, synchronous (3 s timeout):** reserve every line or nothing, using the same `FOR UPDATE ORDER BY product_id` locking as M2.
   - `409` out of stock → undo step 1 (order deleted, cart restored) → answer 409 with the product name.
   - Not running (connection refused) → undo step 1 → `503`: we *know* nothing was reserved.
   - Timeout / 5xx → outcome unknown → cancel + release (Q58) → 503.
3. Publish `OrderCreated` v2 (PENDING, carries the payment method) and answer `201 PENDING`.
4. **Payment** charges once → `PaymentSucceeded` / `PaymentFailed`.
5. **Order** sets CONFIRMED or CANCELLED (cart restored) → `OrderConfirmed` / `OrderCancelled`.
6. **Inventory** sells or releases the reservation; **Notification** writes the bell message; **Analytics** updates the daily sales.

No transaction spans these steps. Each step is atomic on its own, and failures are repaired by explicit compensations (delete the order, restore the cart, release the stock). That is the idea of a **saga**, which M7 formalises.

58 A call to the inventory service times out. Did it reserve the stock? What do you do?

You can't know: the request may never have arrived, may have succeeded with the answer lost, or may *still be running*. That third outcome is what makes distributed systems hard. So the HTTP client distinguishes "unreachable" (definitely not processed) from "timeout/5xx" (unknown). On unknown, the order service cancels the order, restores the cart, frees the idempotency key, sends a direct `release` and publishes `OrderCancelled` (handled even if inventory is down right now). The dangerous race: the slow reserve arrives *after* the release and reserves stock for a dead order. The fix is a **tombstone**: release of an order inventory has never seen writes a `RELEASED` row, and a later reserve for it gets `409 reservation_released`. A test reproduces exactly this with a deliberately slow proxy (reserve arrives after 3.5 s): order cancelled, late reserve refused, stock unchanged.

59 How do you make "charge the card" safe when the event can arrive twice?

- Check first: if a payment row for this order exists, return it — no provider call.
- Call the provider *outside* any DB transaction (never hold locks while waiting seconds on an external system), passing an **idempotency key = orderId**, so the provider itself refuses a second charge if we crash after charging but before saving.
- `UNIQUE(order_id)` on payments is the backstop: if two deliveries race past the check, the second insert fails and we return the first row.
- Then publish the result in "must" mode: if publishing fails, the handler throws, the event is redelivered, the saved payment is found, and the result is announced again.

Tested: the same `OrderCreated` delivered three times → one payment, one notification, stock sold once. A DLQ replay through the whole chain also charged nobody twice.

60 What does an API gateway do, and what must it not do? Why do services still check the token?

Mine does: routing by path prefix (a table of 8 routes), edge authentication (reject a missing/invalid JWT before it costs a service anything; admin routes need the ADMIN role), the global per-IP rate limit (shared counters in ScaleKV), `X-Request-Id` correlation, CORS, blocking `/internal/*`, 502/504 instead of hanging, and aggregation (`/health/ready` and Admin › Services combine every service's health). Bodies are streamed, never buffered. It must **not** contain business logic (pricing, stock rules, order states) — otherwise it becomes the new monolith that every team must change. Services still verify the JWT (zero trust / defence in depth): anything that reaches a service directly, bypassing the gateway, must not find an open door.

61 Shared database vs database per service — what did you choose?

Honestly: one shared PostgreSQL, with table ownership by convention (inventory writes only `inventory` / `reservations`, payment only `payments`, …), and no foreign keys from service tables to other services' tables. The Order Service keeps its **own copy** of the payment outcome (`orders.payment_status`) instead of reading the payments table. The remaining shortcut: catalog pages still JOIN `inventory` to show stock — a read across a boundary. Database per service gives independent schema changes, scaling and failure isolation, but costs joins and local transactions. The fixes for cross-service reads are events plus a local copy (inventory publishes `StockChanged`, the catalog keeps a read model), API composition, or CQRS. I documented the trade-off rather than pretending the split is complete.

62 The order is saved but `OrderCreated` was lost. How does the system recover, and how would you prevent it?

Two safety nets, both based on "ask the source of truth, then repair":

- **Reconciler** (order service, every 10 s): orders PENDING for more than 20 s are re-reserved (idempotent: "already reserved") and `OrderCreated` is re-published. Payment finds its saved payment, so this also heals a lost `PaymentSucceeded`. Per-order exponential backoff (20 s → 5 min), and orders in backoff are excluded in the query so hopeless ones can't starve new ones.
- **Reservation sweeper** (inventory service, every 30 s): reservations HELD for more than 2 min → ask the order service `GET /internal/orders/:id/status` → CONFIRMED: commit; CANCELLED or missing: release; PENDING: leave.

Inside consumers, "update then publish" uses "must" mode so a failed publish is retried with the event. The real prevention is the **transactional outbox** (M7) or CDC (M8); sweepers stay useful even then, for anything stuck.

63 How do you follow one request across five processes?

With a correlation id. The gateway gives every request an `X-Request-Id` (or keeps a valid one from the client) and returns it in the response. Each service stores it in an `AsyncLocalStorage` context (a "thread-local" for async code), sends it on with every HTTP call it makes, and copies it into the headers of every event it publishes; consumers restore it from the event headers. A test places an order with `X-Request-Id: checkout-trace-42` and asserts that `OrderCreated`, `PaymentSucceeded` and `OrderConfirmed` — published by two different services on two topics — all carry it. In M9 this becomes distributed tracing (OpenTelemetry: trace and span ids, timings per hop).

64 What is the strangler fig pattern?

Migrating a monolith gradually: put a gateway (façade) in front, cut out one capability at a time into a service, route its traffic there, and keep shipping — no big-bang rewrite. The name comes from a vine that grows around a tree until it replaces it. In ScaleCart the gateway went in front of the monolith, payment/inventory/notification were extracted, and what remains *is* the Order Service. The Angular app noticed nothing: same address, same API shapes (the order DTO kept its `payment` field even though the data now comes from events).

65 What new failure modes did microservices add that the monolith didn't have?

- **Partial failure:** one service down while others are up (Q55).
- **Unknown outcomes:** timeouts where the work may or may not have happened (Q58).
- **Duplicates and reordering:** at-least-once delivery, re-publishing, DLQ replays.
- **Eventual consistency:** the order is PENDING for a moment; the UI must show that state.
- **Lost messages:** dual writes between a database and the broker (Q62).
- **Contracts:** event schemas must evolve compatibly (v1 and v2 `OrderCreated` both live in the log).
- **Operations:** seven processes to start, observe and debug; correlation ids become essential.
- **Security at the seams:** service identity (`/internal` token), and proxies changing what "client IP" means (Q69).

LESSONS FROM BUILDING AND REVIEWING M6

66 Why did you make event ids deterministic?

"Order X was confirmed" is one fact, however many times it is published. In M6 events are re-published on purpose (the reconciler, "must" mode retries). With random ids, each re-publish would look like a *new* event to the idempotent consumers, and the customer would get two "Order confirmed" messages and revenue would count twice. So `eventId = hash(type, orderId)`, formatted as a UUID (version-5 style) to fit the `uuid` column: the same fact always has the same id, and `once()` recognises the duplicate. Events that can legitimately happen several times per order would need a sequence number in the hash.

67 How did you handle old events after changing an event's schema?

In M5 `OrderCreated` (v1) was published after payment with a final status. In M6 `OrderCreated` v2 is published at placement, status PENDING, with the payment method. Old v1 events stay in the log forever and can come back through a DLQ replay, so every consumer handles both: a helper `outcomeOf(event)` maps v1 CONFIRMED/CANCELLED and v2 `OrderConfirmed` / `OrderCancelled` to one "how did this order end?" answer, and the payment consumer ignores v1 (those orders were paid inside the monolith). Rules I followed: add fields, never change their meaning; bump `version`; consumers tolerate unknown types; new consumer groups start at offset 0 and must skip history they shouldn't act on.

68 How do you test a distributed system?

I wrote a `test-cluster` helper that starts the *whole* system inside the test process: every service as a real HTTP server on a random free port, the gateway in front, a real ScaleKV and a real ScaleLog broker, and the consumer groups. Every test goes through the gateway, services call each other over real HTTP and events go through the real broker; only the process boundary is missing. The helper can `stop()` and `restart()` a single service, which is how the "service down" tests work. Failure injection: a slow proxy for timeouts, disconnecting the broker for lost events, re-publishing for duplicates, starting the payment group late for "payment service down". Tests wait for asynchronous outcomes by polling (`settle()`, `eventually()`) instead of sleeping. Result: 117 backend tests, run repeatedly to catch flakiness, plus a live run of all seven processes and a headless-Edge browser journey (10/10).

69 Tell me about bugs the code review and tests found in M6.

The review found 10 issues; three good stories:

- **Spoofed IP behind a proxy.** The gateway *appended* to the client's own `X-Forwarded-For`. The services trust the gateway (loopback), so a client sending `X-Forwarded-For: 1.2.3.N` with a new N each time looked like a new IP per request and bypassed the 10-per-minute login limit — unlimited password guessing. Fix: the gateway *replaces* the header with the one address it trusts; a regression test sends 11 logins from 11 fake IPs and expects a 429.
- **Stock held forever.** After a reserve timeout, the stock came back only through a best-effort release and a best-effort event; if both were lost, nothing ever released it. Fix: the inventory sweeper (Q62).
- **Doubled cart.** A cancelled order's lines were *added* back to the cart; if the customer had re-added the same items meanwhile, quantities doubled. Fix: `max(cart, ordered)`.

And one found by the new tests: raw SQL wrote `now()` into a `timestamp` column. The database runs in Asia/Calcutta (+5:30) but Prisma stores UTC, so every reservation looked 5.5 hours younger and the sweeper never touched it. Fix: write `now() AT TIME ZONE 'UTC'`. Lesson: one time-zone convention everywhere, or use `timestamptz`.

70 What is still missing, and what would you do next?

- **Shared database** → database per service with read models (Q61).
- **Best-effort publish + sweepers** → transactional outbox (M7) or CDC (M8).
- **Payment succeeded for an already-cancelled order** (a rare race) is only logged as "refund needed" → a saga with a `RefundPayment` compensation (M7).
- **The reserve call** has a timeout but no retries or circuit breaker → retries with backoff + jitter, a breaker, bulkheads (M7).
- **One shared secret** for service calls → mTLS or signed service tokens.
- **Every order-service instance runs the reconciler** → leader election or a lock (harmless today only because every step is idempotent).
- **Static service URLs** → service discovery and load balancing across instances.
- **Observability** → metrics, tracing and lag alerts (M9).

Being able to list your own system's gaps, and why each was acceptable for now, is usually worth more in an interview than claiming it is complete.